Exam Submission Report: JWT-Based Authentication & Multi-Role API Gateway

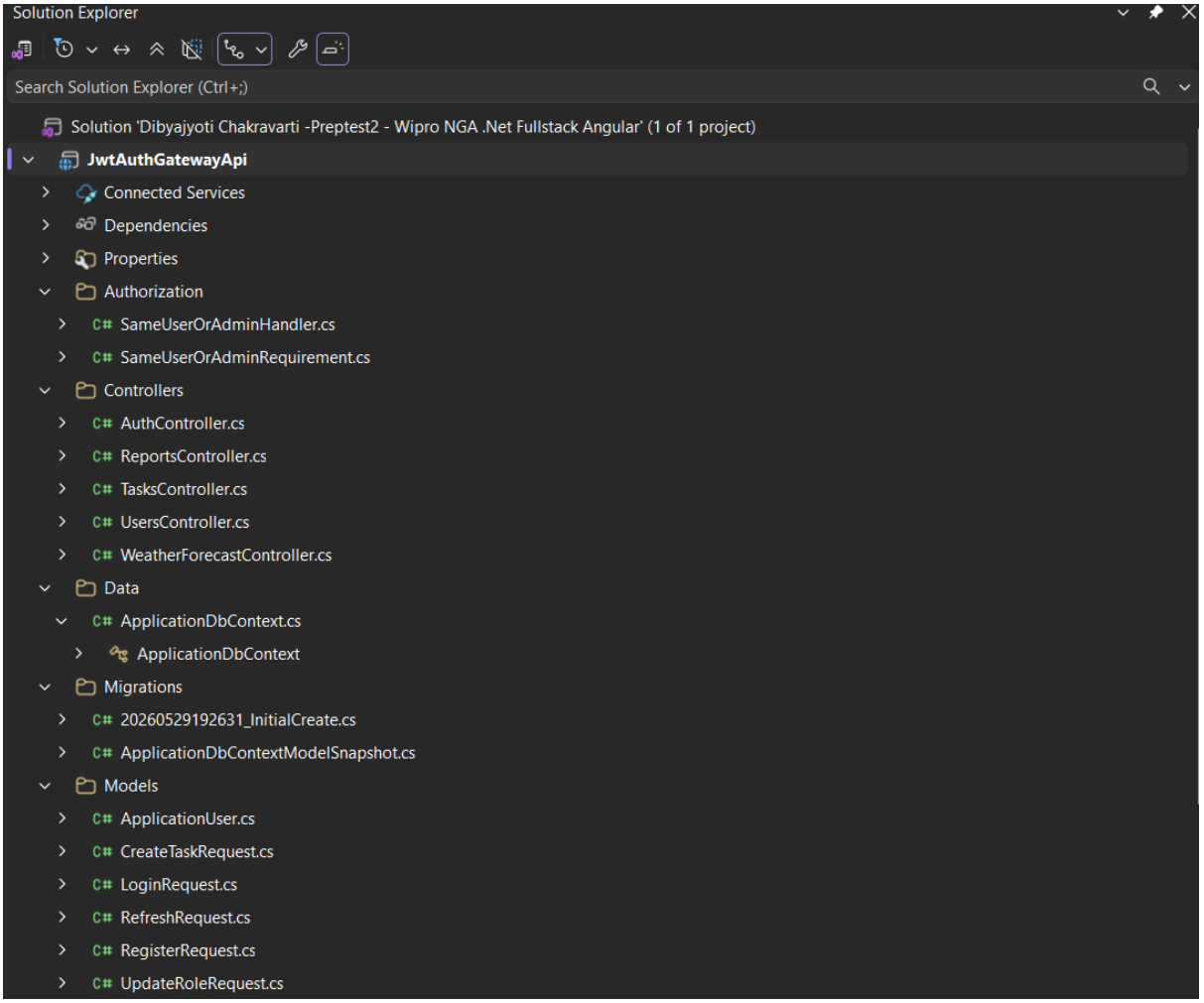
- **Candidate Name:** Dibyajyoti Chakravarti
- **Date:** May 2026
- **Technology Stack:** ASP.NET Core Web API, Entity Framework Core, SQL Server, ASP.NET Core Identity

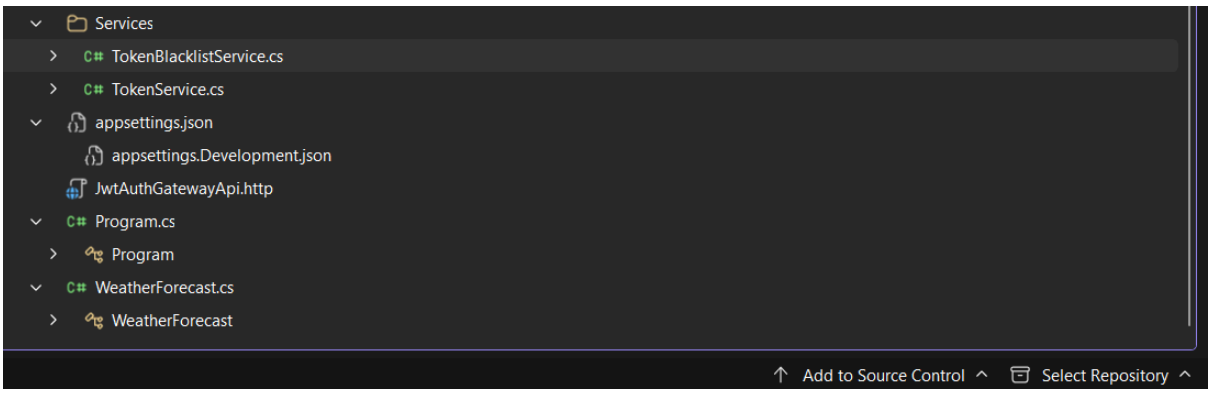
1. Project Overview & Core Architecture

This project implements a secure, centralized API Gateway designed for a multi-tenant SaaS platform. It utilizes JWT-based authentication paired with structured role-based and policy-based authorization layers to strictly manage endpoint access.

Core Capabilities Built:

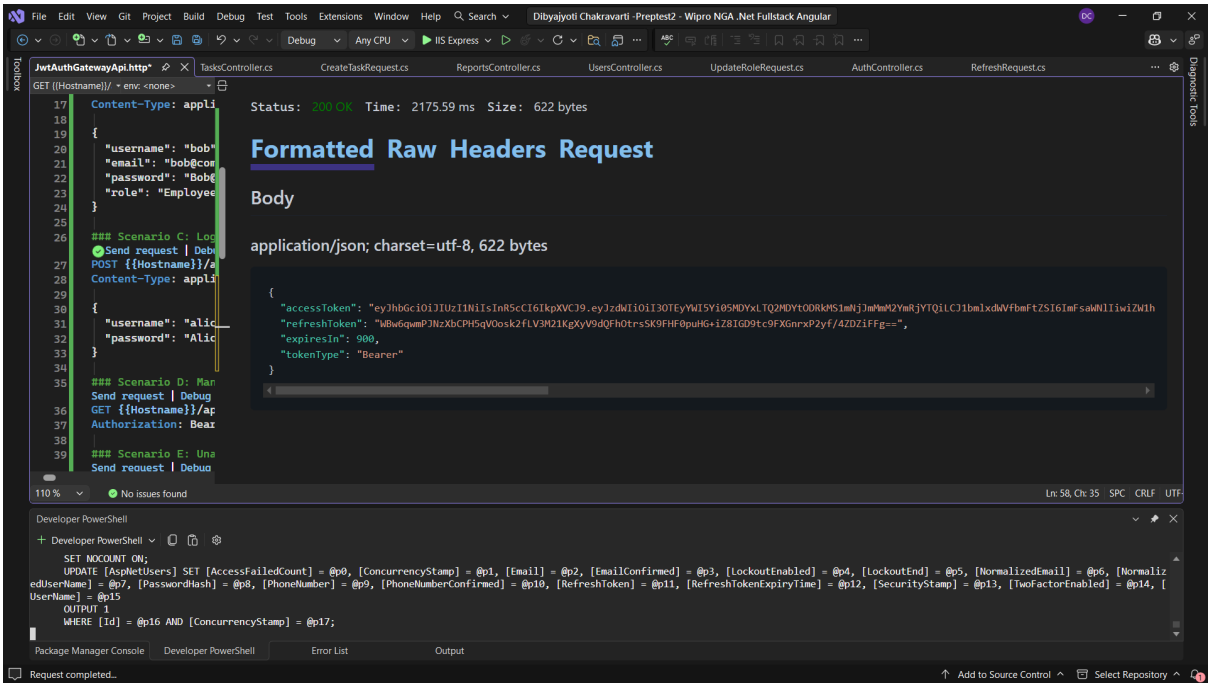
- **Multi-Role RBAC:** Supports automated differentiation across three distinct structural tiers: **SuperAdmin**, **Manager**, and **Employee**.
- **Token Lifecycle Management:** Provides endpoints for user registration, secure login validation, cryptographic token refresh rotation, and stateful session logouts.
- **Security Hardening:** Implements algorithmic brute-force account lockout protection, in-transit HTTPS validation, and centralized CORS policies.
- **Vulnerability Mitigation:** Employs an active runtime token validation pipeline to eliminate token replay attacks and utilizes the ASP.NET Core Data Protection API to encrypt stored user session artifacts.





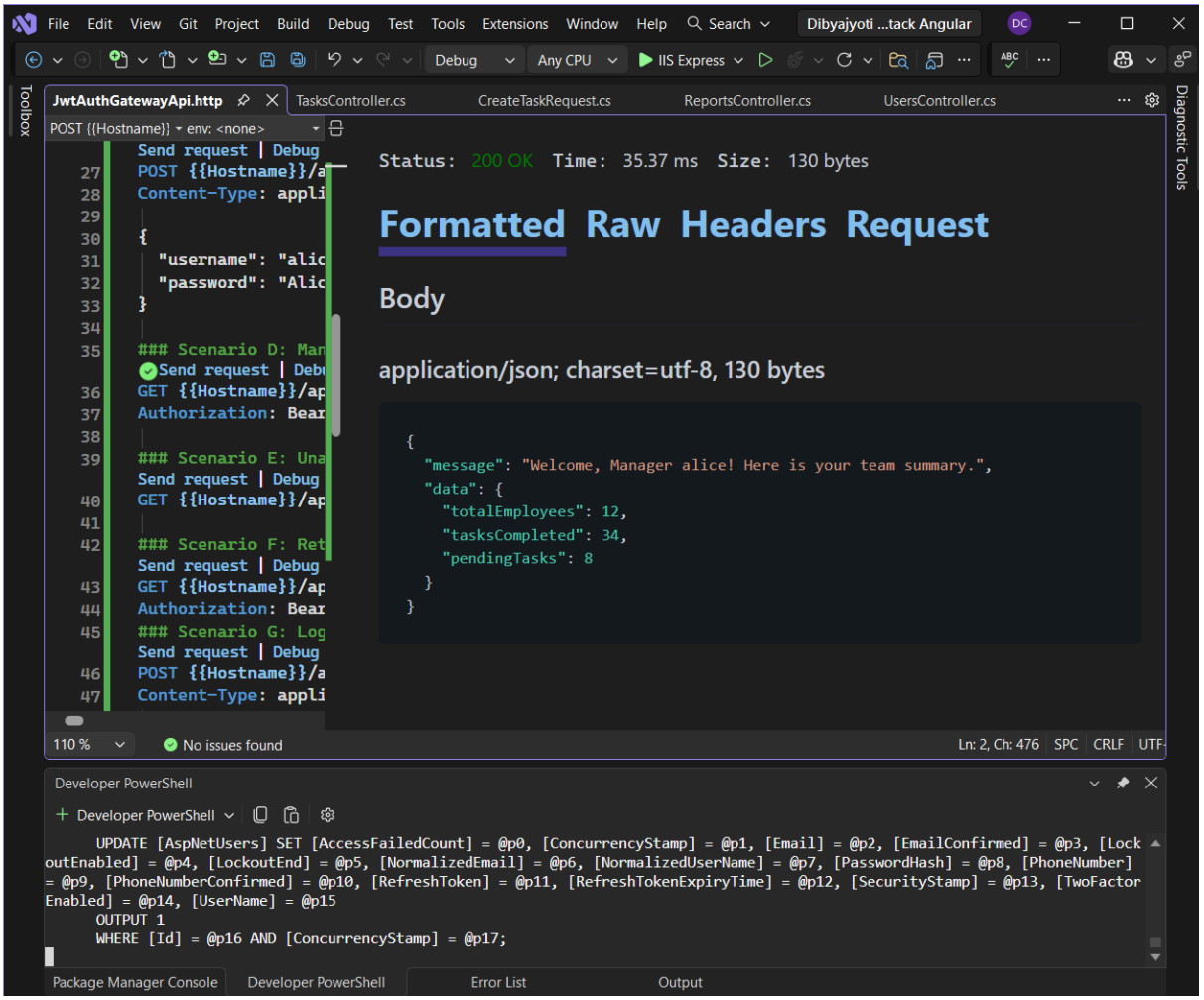
Scenario A & C: Authentication & Token Generation

- **Endpoint:** POST /api/auth/login
- **Verification:** Validates user credentials using the Identity configuration and returns a structured 15-minute access token alongside an encrypted refresh token.



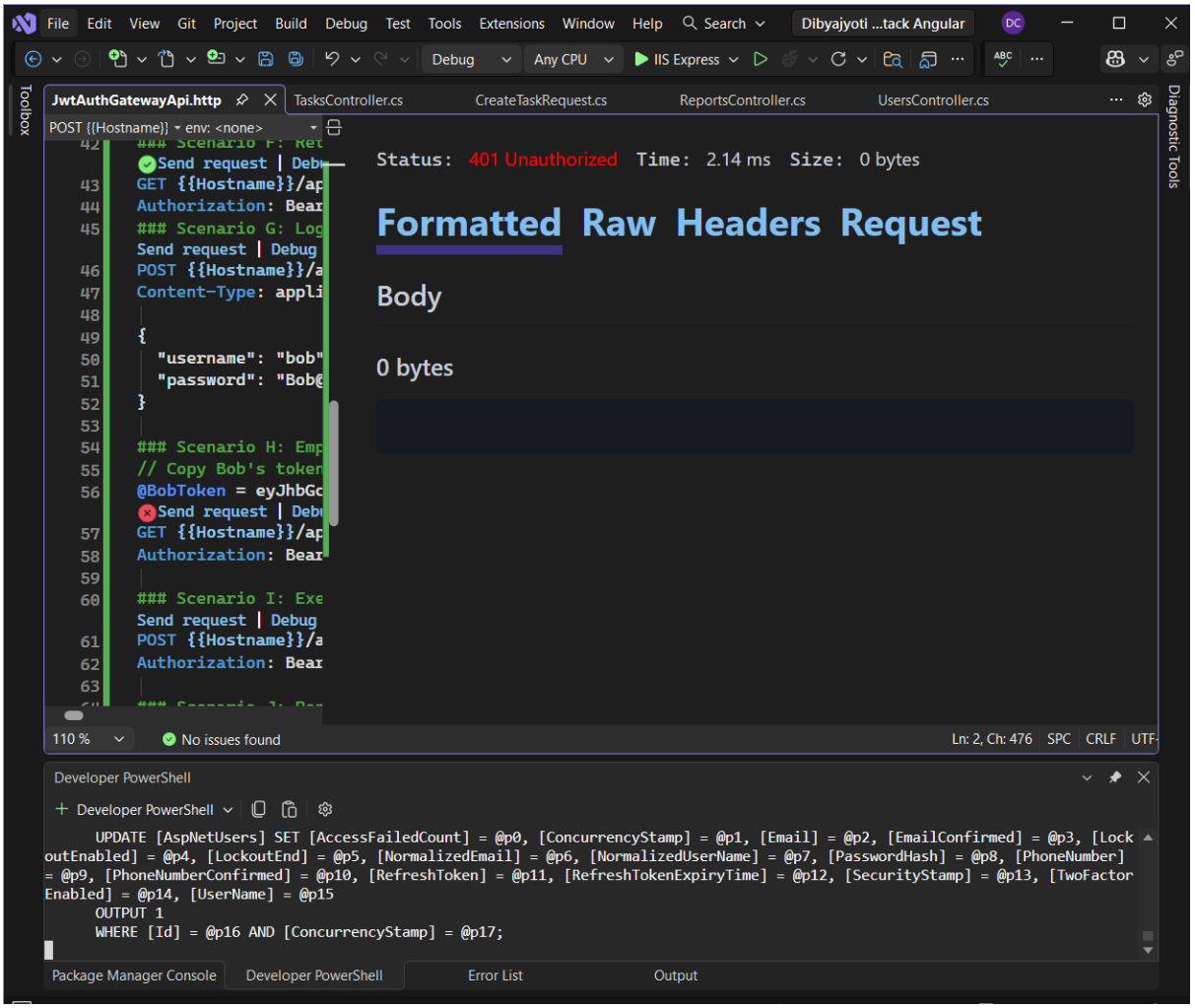
Scenario D: Policy-Based Authorization Validation

- **Endpoint:** GET /api/reports/team-summary
- **Explanation:** Evaluates the custom ManagerOrAbove policy to grant a user with the Manager role secure access to aggregated team report metrics.
- **Visual Evidence:**



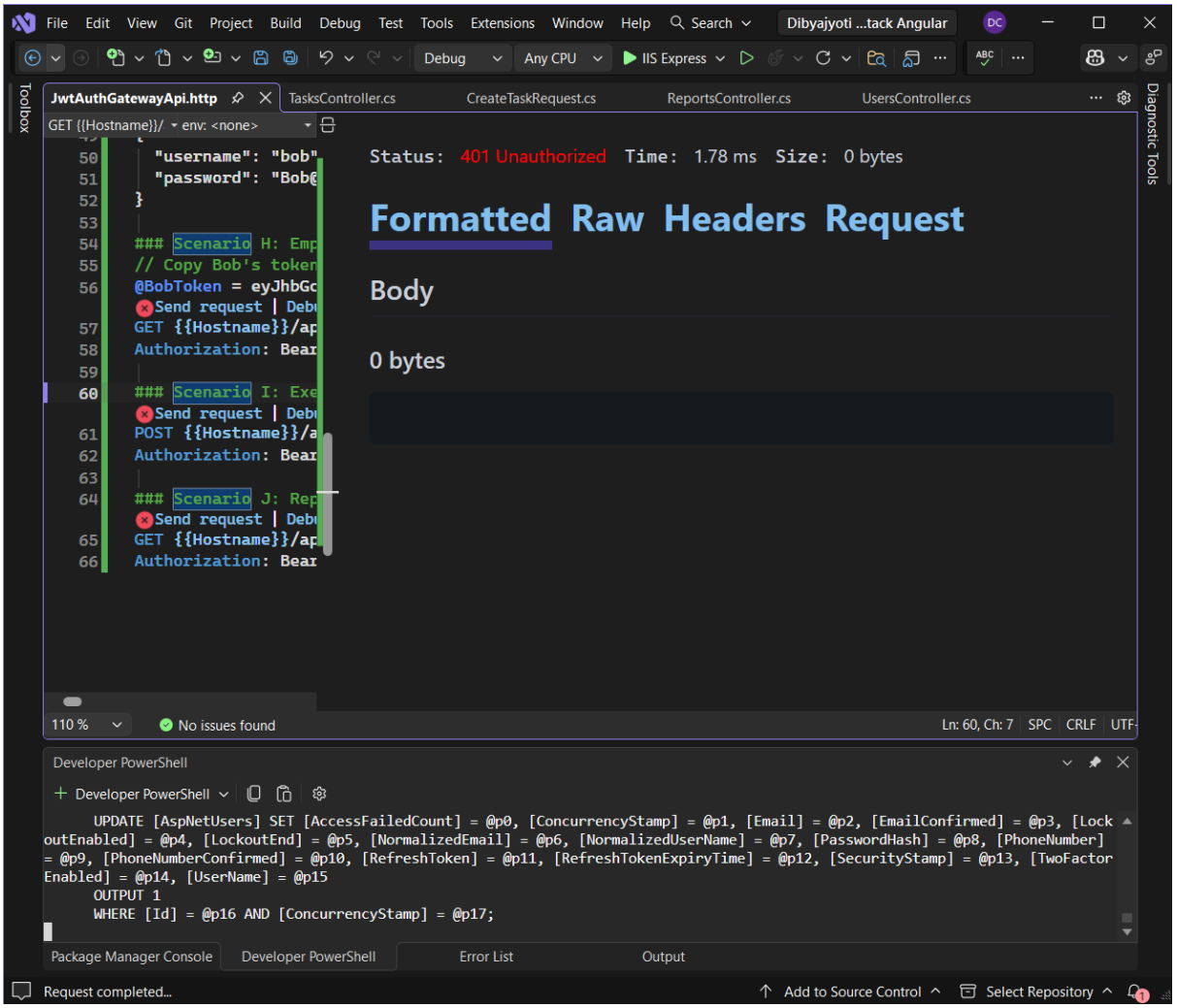
Scenario H: Role Restriction & Error Interception

- **Endpoint:** GET /api/reports/team-summary (Accessed via Bob's Employee Token)
- **One-Liner Explanation:** Intercepts unauthorized low-privilege accounts and explicitly rejects them with a custom structured 403 Forbidden payload.
- **Visual Evidence:**



Scenario I & J: Token Blacklisting & Session Eviction

- **Endpoint:** POST /api/auth/logout
- **One-Liner Explanation:** Tracks the logged-out token's unique ID (jti) in a runtime memory cache to instantly block any token replay exploits with a 401 status.
- **Visual Evidence:**


File Edit View Git Project Build Debug Test Tools Extensions Window Help Search
Debug
Any CPU
IIS Express
JwtAuthGatewayApi.http
TasksController.cs
CreateTaskRequest.cs
ReportsController.cs
UsersController.cs
GET {{Hostname}}/ env: <none>
"username": "bob"
"password": "Bob@
}
Scenario H: Emp
// Copy Bob's token
@BobToken = eyJhbGc
Send request | Debu
GET {{Hostname}}/ap
Authorization: Bear
Scenario I: Exe
Send request | Debu
POST {{Hostname}}/a
Authorization: Bear
Scenario J: Rep
Send request | Debu
GET {{Hostname}}/ap
Authorization: Bear
Status: 401 Unauthorized Time: 1.78 ms Size: 0 bytes
Formatted Raw Headers Request
Body
0 bytes
110 %
No issues found
Ln: 60, Ch: 7 SPC CRLF UTF
Developer PowerShell
Developer PowerShell
UPDATE [AspNetUsers] SET [AccessFailedCount] = @p0, [ConcurrencyStamp] = @p1, [Email] = @p2, [EmailConfirmed] = @p3, [LockoutEnabled] = @p4, [LockoutEnd] = @p5, [NormalizedEmail] = @p6, [NormalizedUserName] = @p7, [PasswordHash] = @p8, [PhoneNumber] = @p9, [PhoneNumberConfirmed] = @p10, [RefreshToken] = @p11, [RefreshTokenExpiryTime] = @p12, [SecurityStamp] = @p13, [TwoFactorEnabled] = @p14, [UserName] = @p15
OUTPUT 1
WHERE [Id] = @p16 AND [ConcurrencyStamp] = @p17;
Package Manager Console
Developer PowerShell
Error List
Output
Request completed...
Add to Source Control
Select Repository